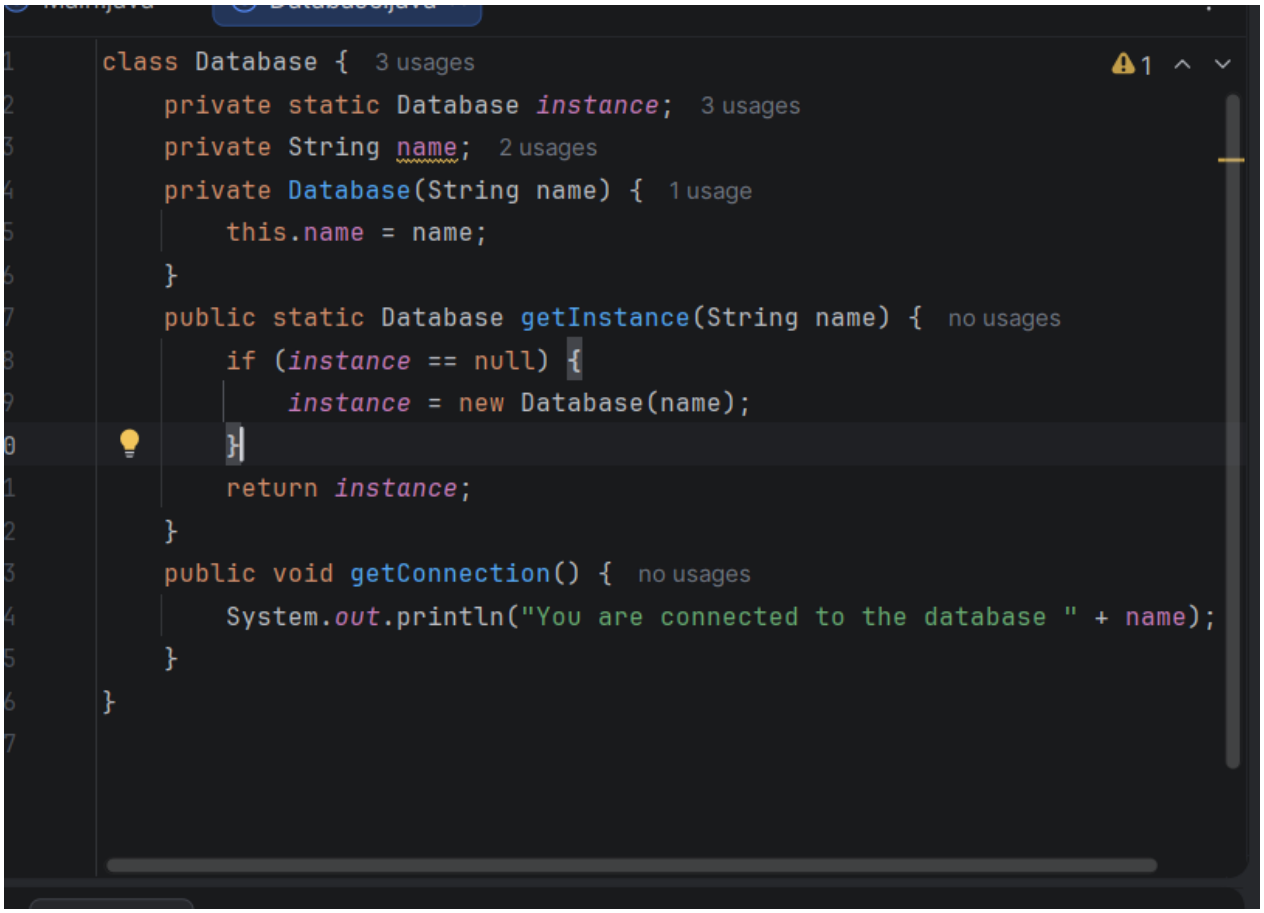
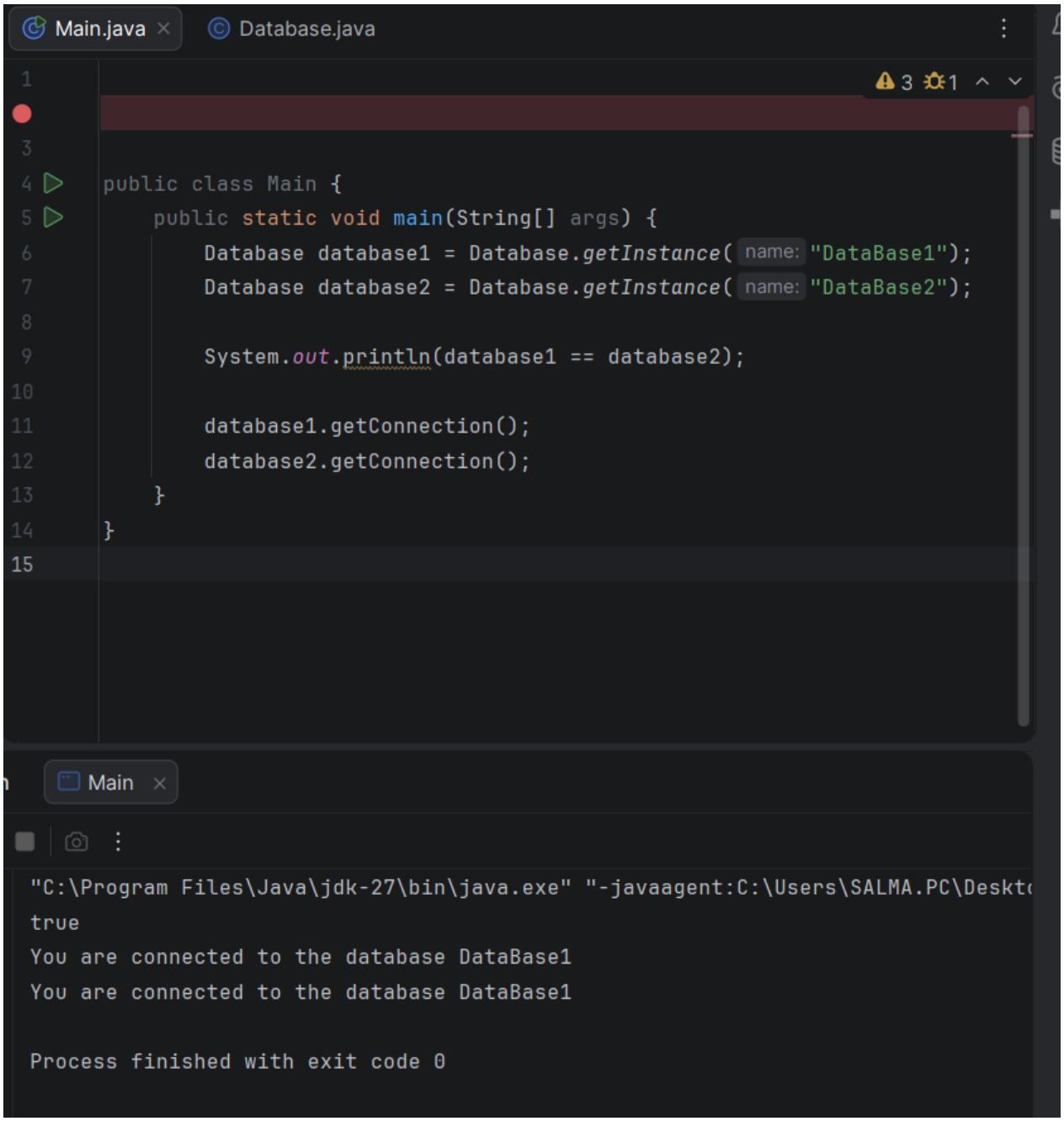
Design Patterns: Factory & Singleton

MANA Salma

Exercice 1 :

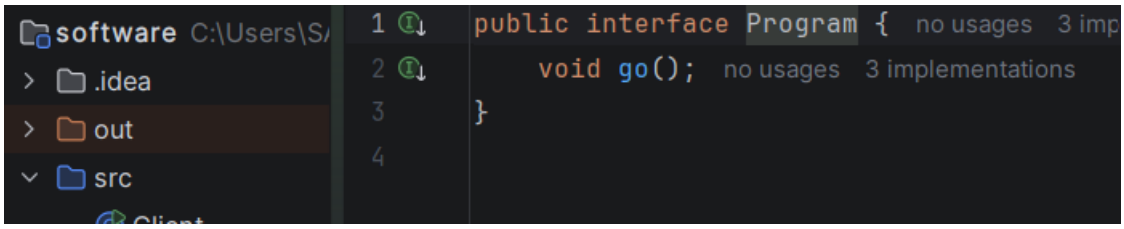


```
class Database {  3 usages
    private static Database instance;  3 usages
    private String name;  2 usages
    private Database(String name) {  1 usage
        this.name = name;
    }
    public static Database getInstance(String name) {  no usages
        if (instance == null) {
            instance = new Database(name);
        }
        return instance;
    }
    public void getConnection() {  no usages
        System.out.println("You are connected to the database " + name);
    }
}
```



The Singleton design pattern ensures that only one instance of the Database class can be created. The private constructor prevents direct object creation, while the getInstance() method creates the instance only once and returns it for every subsequent request. In the main() method, we create two references using different names and compare them with ==. The result is true, confirming that both references point to the same instance, which retains the name DB1.

Exercice 2 :




```
public class Program1 implements Program {  no usages
    public void go() {  no usages
        System.out.println("Executing Program 1");
    }
}
```

```
public class Program2 implements Program {  no usages
    public void go() {  no usages
        System.out.println("Executing Program 2");
    }
}
```

```
public class Program3 implements Program {  no usages
    public void go() {  no usages
        System.out.println("Executing Program 3");
    }
}
```

Instead of choosing a program using conditions, the client receives the program to execute.

```java
public class Client {
    public void execute(Program p) {  3 usages
        p.go();
    }
    public static void main(String[] args) {
        Client c = new Client();
        c.execute(new Program1());
        c.execute(new Program2());
        c.execute(new Program3());
    }
}
```



What are the advantages?

- Less coupling: Client depends on the Program interface, not on specific program classes.
- Less code duplication: the execute() method is written only once.
- Easy extension: we can add Program4 without modifying Client.
- Flexibility: we can choose different programs at runtime.

Conclusion: The Strategy pattern makes the code more flexible, reusable, and easier to maintain.

Class diagram :

